Revisiting the Role of Feature Engineering for Compound Type Identification in Sanskrit

Jivnesh Sandhan, Amrith Krishna*, Pawan Goyal* and Laxmidhar Behera

Department of Electrical Engineering, IIT Kanpur
* Department of Computer Science and Engineering, IIT Kharagpur

October 24, 2019

Introduction

Compound

Combination of more than one word into one word where generated word is treated as a fully qualified word (*pada*).

- Highly productive.
- It is single word with single accent.
- The compound formation is binary with an exception of *Dvandva* and *bahupada bahuvrīhi*
- Order of components matters.
- Subject to all the inflectional and derivational changes applicable to nouns

Compound Type Identification Problem

- Generally treated as word level semantic task in NLP.
- It's multi-class classification.
- *Aṣṭādhyāyī*: semantically four main categories of compounds.
- Similar to prior computational approaches in Sanskrit compounding, we chose no of classes to be 4.
- It is an essential pre-processing step for improving on overall downstream NLP applications.

Category	**Example**	**Meaning**
Avyayībhāva	*yathāśakti* (यथाशक्ति)	as per ability
Bahuvrīhi	*pācikābhāryaḥ* (पाचिकाभार्या)	person whose wife is cook
Tatpuruṣa	*daśarathaputraḥ* (दशरथपुत्रः)	son of dasharatha
Dvandva	*rāmalakṣmaṇau* (रामलक्ष्मणौ)	Ram and Lakshmana

Table: Broad categories of compounds in Sanskrit

Challenges involved in compound type identification

- *Nañ-Tatpuruṣa* vs *Nañ-Bahuvrīhi*
  - *ajñānaṃ* (अज्ञानं) belongs to Tatpuruṣa and akāraṇaḥ (अकारणः) belongs to Bahuvrīhi. Both has same negation component.
- *Tatpuruṣa* vs *Avyayībhāva*
  - *upajīvataḥ* (उपजीवतः) has the first component as avyaya which is strong characteristic of Avyayībhāva but it belongs to Tatpuruṣa.
- Compounds with same components but different possible classes
  - *aputraḥ* (अपुत्रः), ativanam (अतिवनं) etc
- It is non-trivial even for humans and depends on the context in which they are used.

Motivation (1/2)

- Neural models have become part and parcel of NLP pipeline for resource-rich languages.
- We focus on using such approaches for low resource languages like Sanskrit.
- Concerted effort in studying the nature of compositionality in compounds by leveraging on distributional word-representations.
- Word embedding as the sole features have shown competitive results compared to feature rich models.
- Recent statistical approach for compound type identification task which combined lexical and distributional information. Effective model but linguistically involved.

Motivation (2/2)

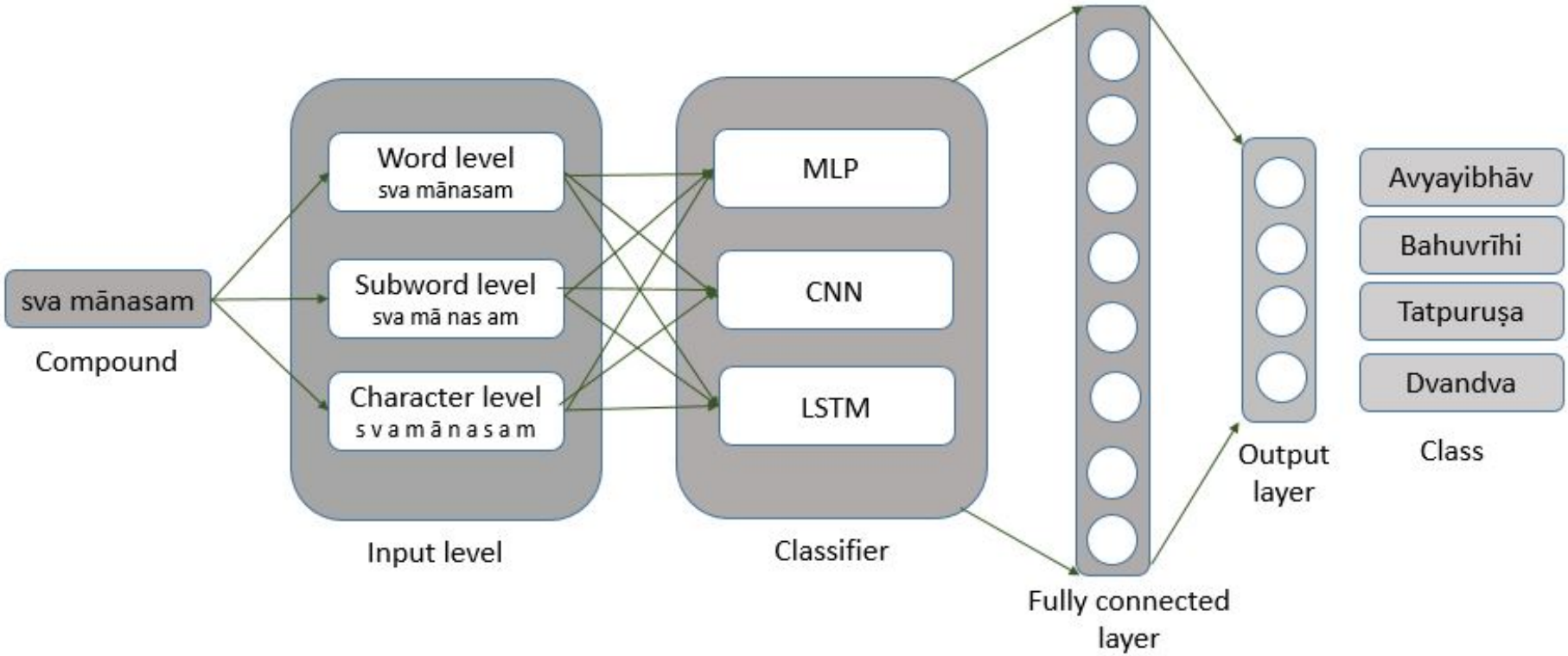
Feature engineering approach[a]

- Features based on rules from *Aṣṭādhyāyī*
- Information based on lexical network *Amarakoṣa*.
- Features based variable length n-grams learned from data using Adaptor grammar.

[a] Amrith Krishna, Pavankumar Satuluri, Shubham Sharma, Apurv Kumar, and Pawan Goyal .2016. Compound type identication in Sanskrit: What roles do the corpus and grammar play? (WSSANLP2016),pages 110

Can recent advances in neural network outperform traditional hand engineered feature based methods on the semantic level multi-class compound classification task for Sanskrit?

System Description



Pipeline flow of our classification framework

Word Representation

One hot encoding.

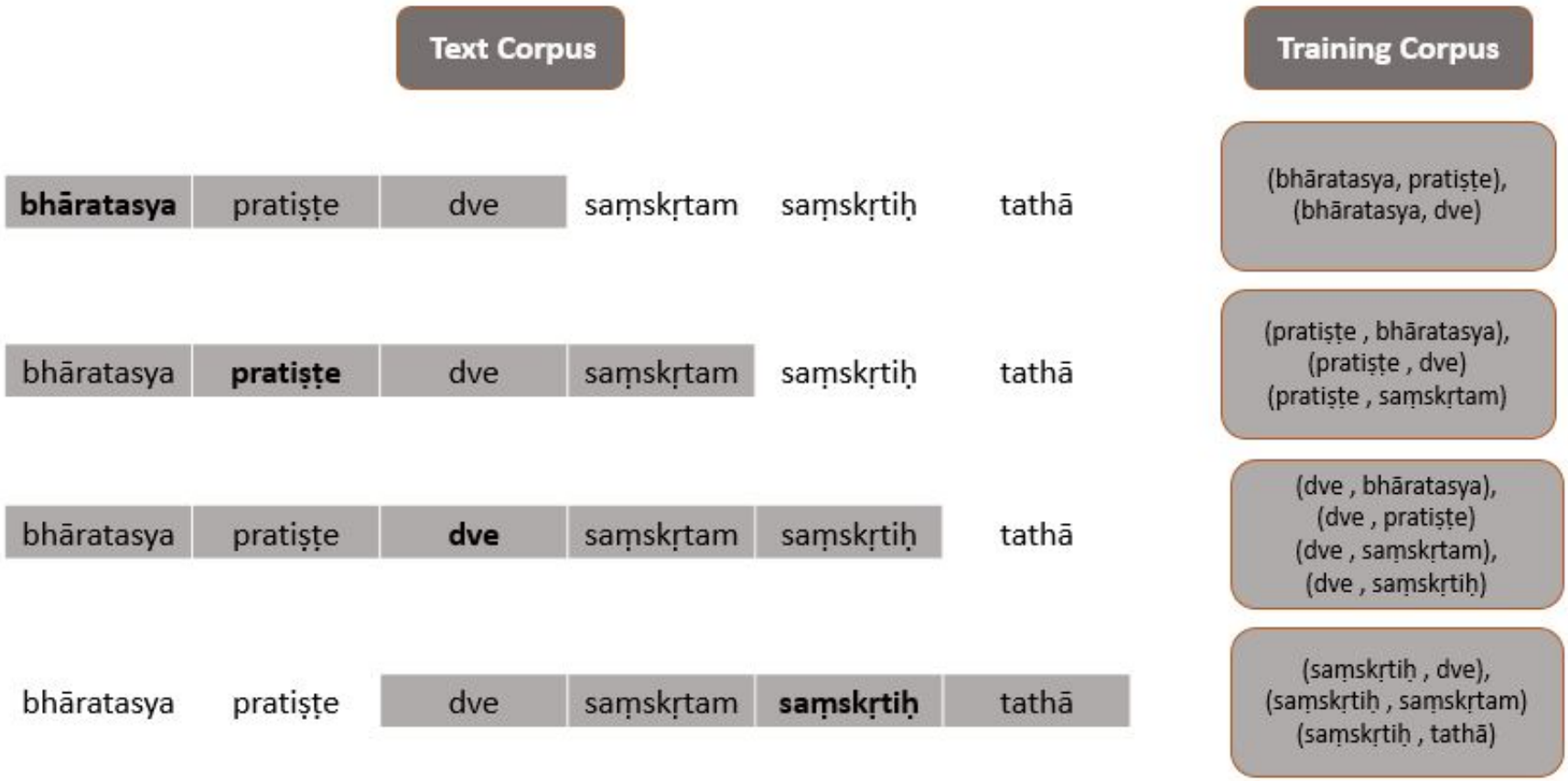
- No natural notion of similarity.
- For example, *naraḥ* (नरः) vs rājā (राजा)

naraḥ	0	0	1	0	0
rājā	1	0	0	0	0

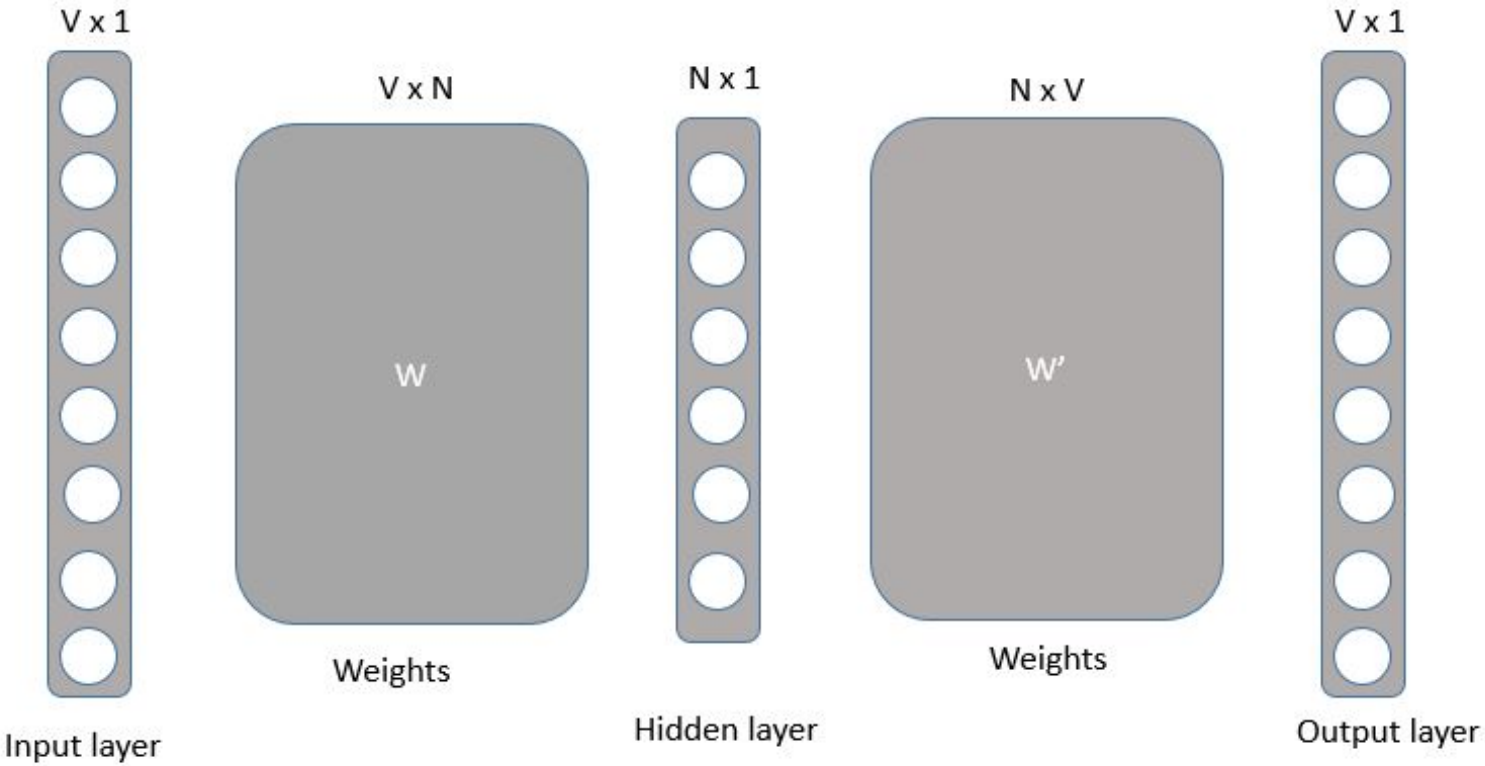
Distributional semantics:

- You know a word by the company it keeps.
- For example, bhāratasya pratiṣṭe dve **saṃskṛtam** saṃskṛtiḥ tathā

How do we learn word representation?



Word2vec architecture



Skip-gram architecture

Dataset

Compound classification dataset

Compound dataset obtained from the department of Sanskrit studies, UoHyd. These compounds are part of ancient texts, namely, *Srīmad Bhagavat Gīta, Caraka saṃhīta*, etc.

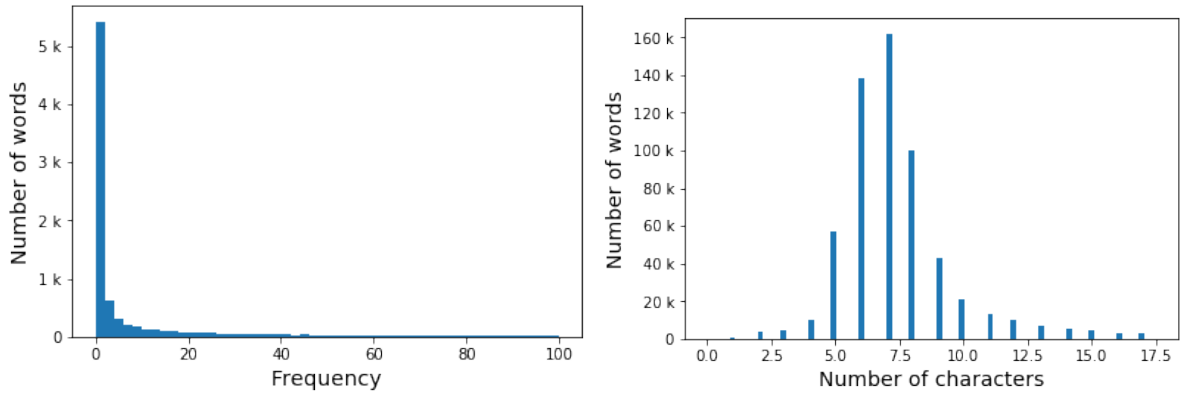
Category	Samples
Avyayībhāva	239
Bahuvrīhi	4,271
Tatpuruṣa	4,266
Dvandva	1,176
Total	9,952

Table: Class-wise samples of compound dataset

Corpus for learning word representations

Our text corpus contains data from the Digital Corpus of Sanskrit (3.8 M), Wikipedia (0.7 M) and Vedabase (0.2 M).

Rare/ non-occurrences of compounds in corpus



(a) (b)

Figure: (a) 50% of compounds have zero occurrence in corpus. (b) Distribution of number of characters per word in the corpus.

Solution: BPE

- Originally a compression algorithm.
- Most frequent byte pair into a new byte
- Maximizes the language-model likelihood of the training data with newly learned vocabulary.

Architectures

Three levels of input to system with embeddings used

- Word level : FastText, FastText*
- Subword level : BPE+Word2Vec, BPE+Glove
- Character level : CharCNN

We learned word embeddings of these components of the compound from our Sanskrit corpus

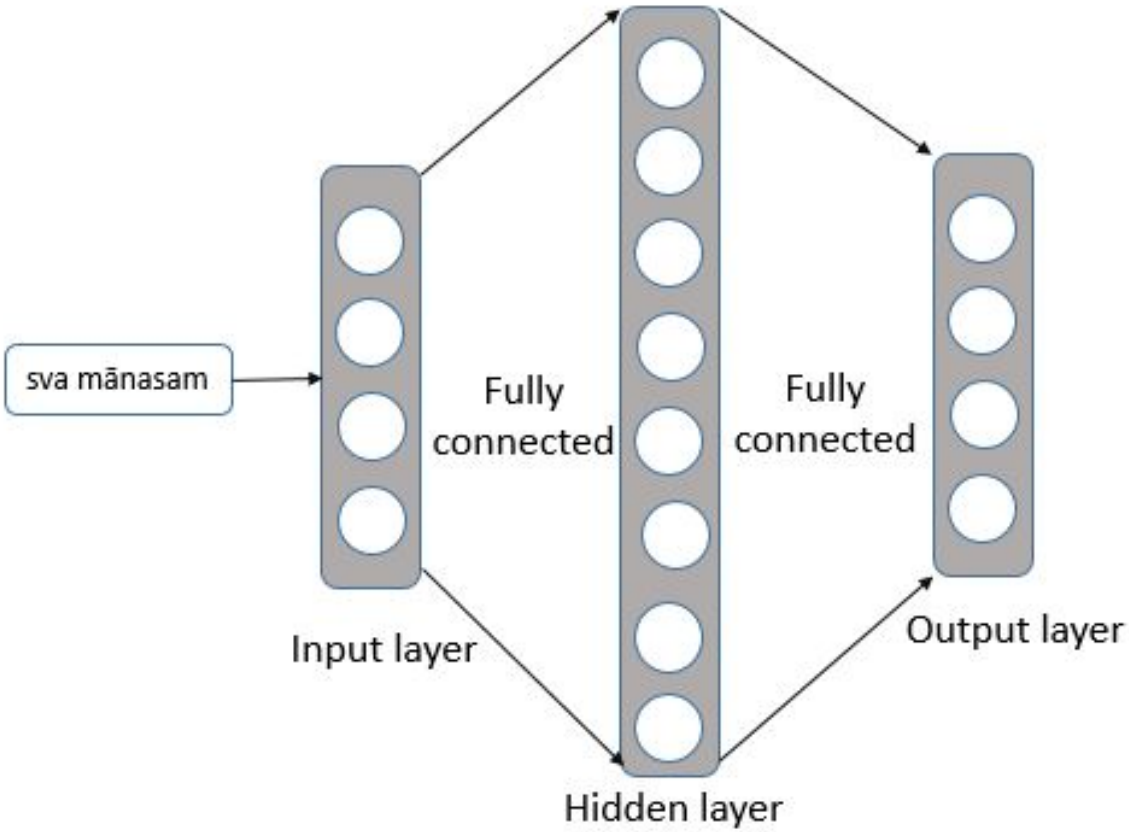
Neural based architectures

For well-organized analysis based on standard neural architectures:

- Multi-Layer Perceptron (MLP)
- Convolution Neural Network (CNN)
- Long Short Term Memory (LSTM)

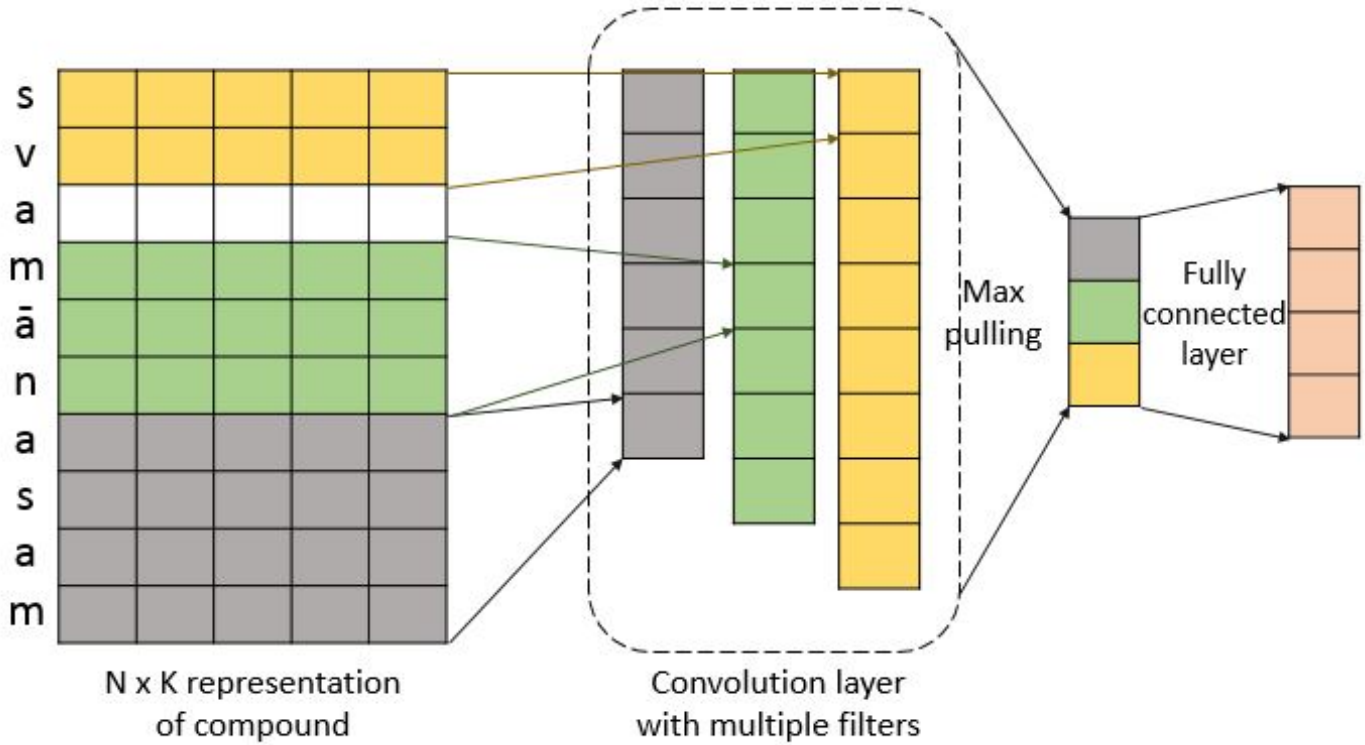
In all the architectures, *relu* activation function for dense layer, *softmax cross entropy* loss function and *adam* optimizer are used.

Architectures: MLP



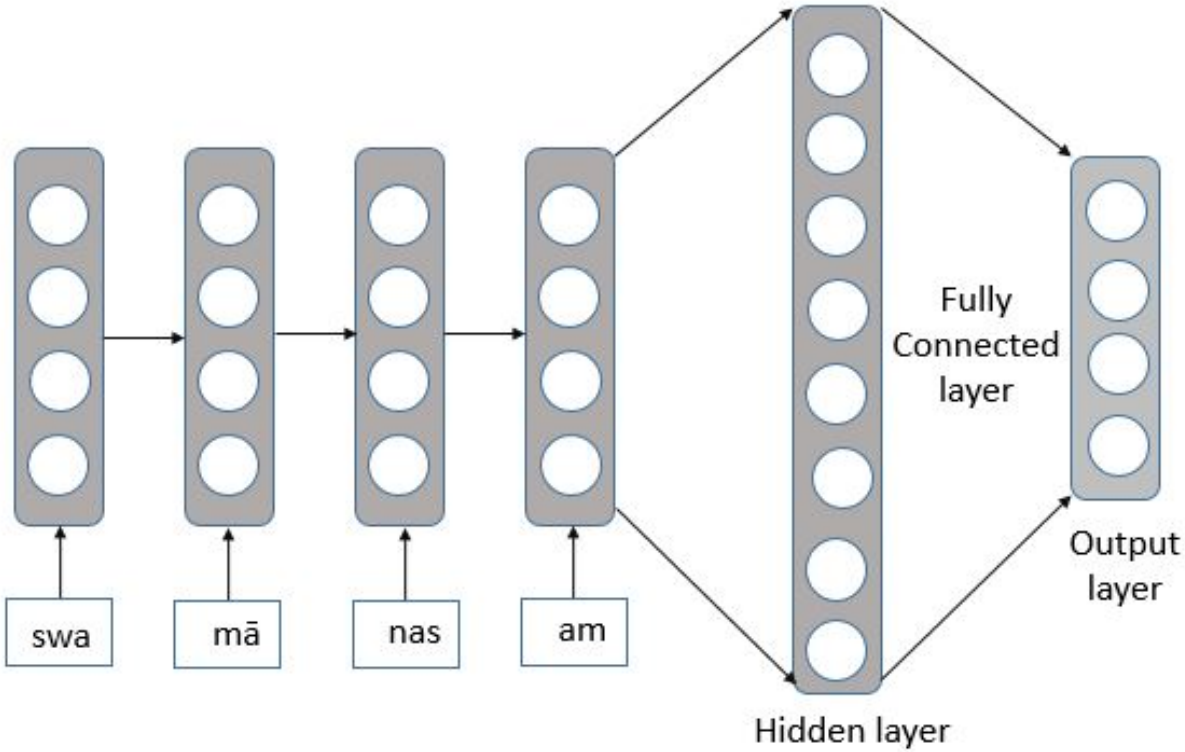
Word level input + MLP based classifier

Architectures: CNN



Character level input + CNN based classifier

Architectures: LSTM



Subword level input + LSTM (RNN) based classifier

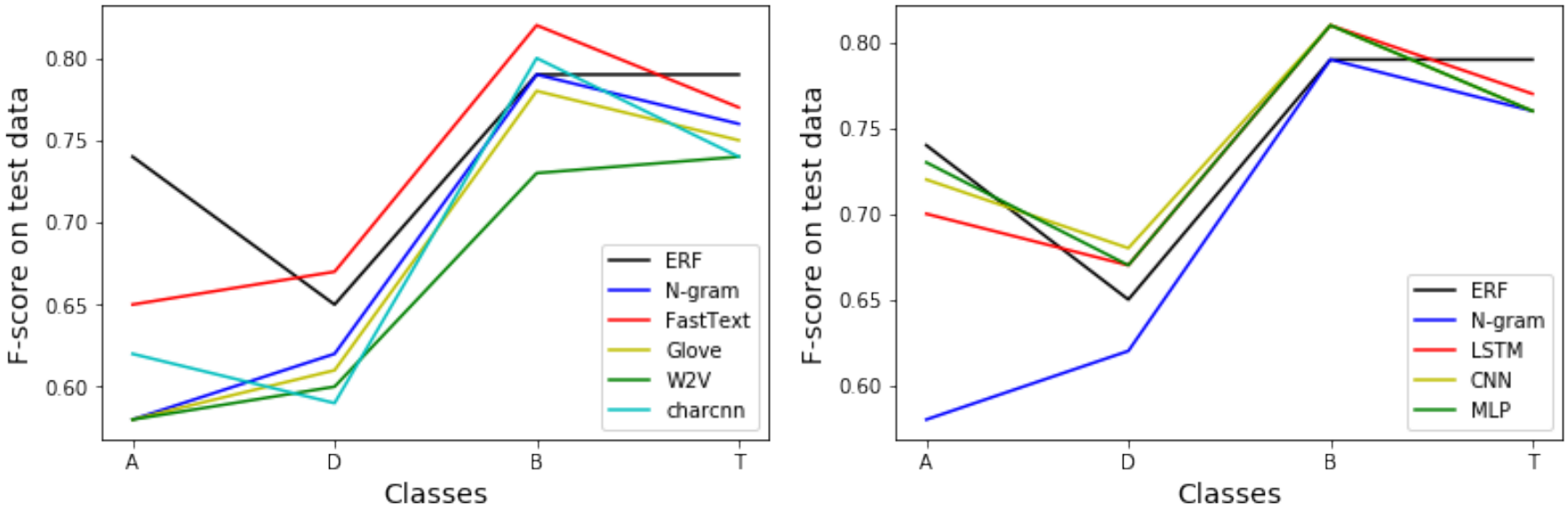
Results

Evaluation metric: accuracy, precision, recall and F-score.

Embedding	Classifier	A	P	R	F
baseline	ERF	77.39	0.78	**0.72**	**0.74**
	RF(N-gram)	75.88	**0.83**	0.64	0.70
Random	CNN+	66.15	0.63	0.57	0.59
charcnn	CNN+	74.65	0.73	0.65	0.68
bpe+w2v	CNN+	71.90	0.74	0.64	0.67
bpe+glove	CNN+	74.13	0.76	0.64	0.68
FastText*	MLP	74.51	0.72	0.66	0.68
FastText	LSTM+	**77.68**	0.76	0.71	0.73

Table: Evaluation measures are micro accuracy (A), macro precision (P), macro recall (R). macro F-score (F). Results reported on the test data are averaged over 5 runs.'+' sign indicates end to end training integrated with classifier.

Results

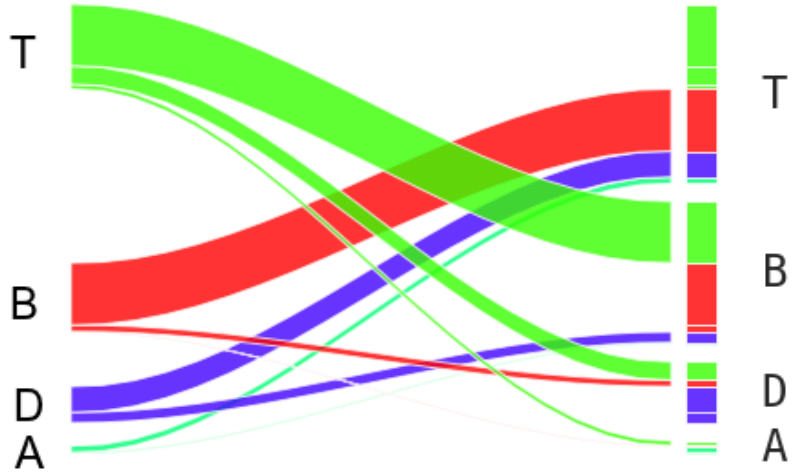


(a) (b)

Figure: (a) Class-wise F-score for different embedding with the same architecture (CNN) (b) Class-wise F-score for different architectures with the same embedding (FastText). Note that class size increases as we move from left to right along x-axis. ERF and N-gram are baselines reported in Table 3.

Error Analysis

(a)



(b)

Figure: (a) Confusion matrix heat-map for our best performing system (A, B, D and T refer to *Avyayībhāva*, *Bahuvrīhi*,*Dvandva*, *and Tatpuruṣa*, respectively) (b) Alluvial graph for showing mis-classification to demonstrate conflicts between classes.

Error Analysis

- No mis-classification between *Dvandva* and *Avyayībhāva*.
- 11 data-points from *Tatpuruṣa* got mis-classified into *Avyayībhāva* where all of them have the first component as *avyaya*.
- Most of the compounds from *Avyayībhāva* misclassified into *Tatpuruṣa*.
- Conflict between *Tatpuruṣa* and *Bahuvrīhi*. More than 600 unique components of compounds are common in training set.
- To solve these ambiguities context is essential but we explored up to what degree the ambiguities can be resolved if we have only string level information.

Conclusion

- For resource-rich languages, deep learning mostly replaced the need for feature engineering with the choice of a good model architecture.
- We experimented with different level of input representations standard neural architectures.
- Our best model gives an F-score of 0.73 compared to the state of the art feature rich baseline (0.74).

Future Work

- What is the effect of the corpus size on the performance? 5 M (ours) vs 84 B (Glove)
- Would a larger dataset have helped?
- Could methods based on cross-lingual embeddings help in this scenario for transfer learning from languages similar to Sanskrit?

Thank You